快速傅氏变换与实验

朱君鹏

长期以来，**快速傅氏变换**(Fast Fourier Transform)和**离散小波变换**(Discrete Wavelet Transform)在数字信号处理、石油勘探、地震预报、医学断层诊断、编码理论、量子物理及概率论等领域中都得到了广泛的应用。各种快速傅氏变换(FFT)，成为数值代数方面最活跃的一个研究领域，而其意义远远超过了算法研究的范围，进而为诸多科技领域的研究打开了一个崭新的局面。

1 快速傅里叶变换 FFT

离散傅里叶变换是 20 世纪 60 年代是计算复杂性研究的主要里程碑之一，1965 年 Cooley 和 Tukey 所研究的计算**离散傅里叶变换**(Discrete Fourier Test)的快速傅氏变换(FFT)将计算量从 O(n^2)下降至 O($n$log$n$)，推进了 FFT 更深层、更广法的研究与应用。FFT 算法有很多版本，但大体上可分为两类：迭代法和递归法，本节仅讨论迭代法。

1.1 串行 FFT 迭代算法

假定 $a[0],a[1],\cdots,a[n\text{-}1]$ 为一个有限长的输入序列，$b[0],\ b[1],\ \cdots,b[n\text{-}1]$为离散傅里叶变换的结果序列，则有：$b[k]=\sum_{m=0}^{n-1}a[k]W_n^{km}(k=0,1,2,\ldots,n-1)$，其中 $\mathrm{W}_n=e^{\frac{2\pi i}{n}}$，实际上，上式可写成矩阵 W 和向量 a 的乘积形式：

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} w^0 & w^0 & w^0 & \cdots & w^0 \\ w^0 & w^1 & w^2 & \cdots & w^{n-1} \\ w^0 & w^2 & w^4 & \cdots & w^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w^0 & w^{(n-1)} & w^{2(n-1)} & \cdots & w^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

一般的 n 阶矩阵和 n 维向量相乘，计算时间复杂度为 n^2，但由于 W 是一种特殊矩阵，故可以降低计算量。

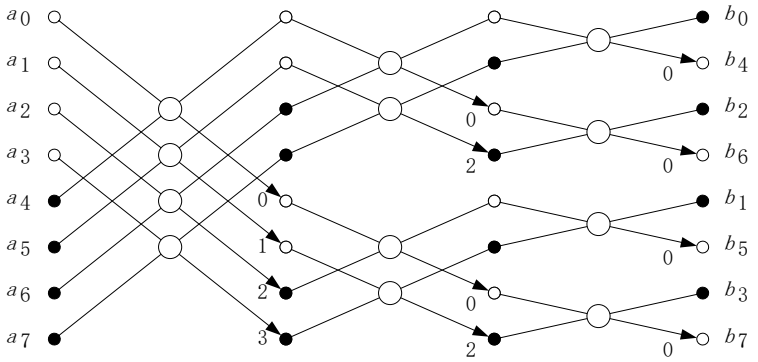

图 1　n=4 时的 FFT 蝶式变换图

显然, FFT 算法的计算复杂度为 O($n$log$n$)。

1.2 并行 FFT 算法

设 P 为处理器的个数，一种并行 FFT 实现时是将 n 维向量 a 划分成 p 个连续的 m 维子向量，这里 $m=\lceil n/p \rceil$，第 i 个子向量中下标为 $i\times m$, …, $(i+1)\times m-1$，其元素被分配至第 i 号处理器（i=0,1, …, p-1）。由图 1 可以看到，FFT 算法由 log$n$ 步构成，依次以 $2^{\log n-1}$、$2^{\log n-2}$、…、2、1 为下标跨度做蝶式计算，我们称下标跨度为 2^h 的计算为第 h 步（h=log$n$-1, log$n$-2, …, 1, 0）。并行计算可分两阶段执行：第一阶段，第 log$n$-1 步至第 log$m$ 步，由于下标跨度 $h\geqslant m$，各处理器之间需要通信；第二阶段，第 log$m$-1 步至第 0 步各处理器之间不需要通信。具体并行算法框架描述如下：

FFT 并行算法

输入： $a=(a_0,a_1, \cdots,a_{n-1})$

输出： $b=(b_0,b_1, \cdots,b_{n-1})$

Begin

对所有处理器 my_rank(my_rank=0,…, p-1)同时执行如下的算法:

(1)**for** h=log$p$-1 **downto** 0 **do**

/* 第一阶段，第 log$n$-1 步至第 log$m$ 步各处理器之间需要通信*/

(1.1) $t=2^{i}$, $l=2^{(i+\log m)}$,$q=n/l$, $z=w^{q/2}$, $j=j+1$,$v=2^{j}$ /*开始 j=0*/

(1.2)**if** ((my_rank mod t)=(my_rank mod 2t)) **then**

/*本处理器的数据作为变换的前项数据*/

(i) tt= my_rank+p/v

(ii)接收由 tt 号处理器发来的数据块，并将接收的数据块存于 c[my_rank*m+n/v]到 c[my_rank*m+n/v+m]之中

(iii)**for** k=0 **to** m-1 **do**

$c[k]=c[k]+c[k+n/v]$

$c[k+n/v]=(c[k]-c[k+n/v])*z^{(\text{my_rank}*m+k) \bmod l}$

end for

(iv)将存于 c[my_rank*m+n/v]到 c[my_rank*m+n/v+m]之中的数据块发送到 tt 号处理器

else /*本处理器的数据作为变换的后项数据*/

(v)将存于之中的数据块发送到 my_rank-p/v 号处理器

(vi)接收由 my_rank-p/v 号处理器发来的数据块存于 c 中

end if

end for

(2)**for** i=log$m$-1 **downto** 0 **do** /*第二阶段，第 log$m$-1 步至第 0 步各处理器之间不需要通信*/

(2.1) $l=2^{i}$,$q=n/l$, $z=w^{q/2}$

(2.2)**for** k=0 **to** m-1 **do**

if ((k mod l)=(k mod 2l)) **then**

$c[k]=c[k]+c[k+l]$

$c[k+l]=(c[k]-c[k+l])*z^{(\text{my_rank}*m+k) \bmod l}$

end if

end for

end for

End

由于各处理器对其局部存储器中的 m 维子向量做变换计算，计算时间为 $m\log n$；点对点通信 $2\log p$ 次，每次通信量为 m，通信时间为 $2(t_s + mt_w)\log p$，因此快速傅里叶变换的并行计算时间为 $T_p = m\log n + 2(t_s + mt_w)\log p$。

附录 FFT 并行算法的 MPI 源程序

1. 源程序 fft.c

```
#include <stdio.h>
#include <stdlib.h>
#include <complex.h>
#include <math.h>
#include "mpi.h"

#define    MAX_PROCESSOR_NUM    12
#define    MAX_N                50
#define    PI                   3.141592653
#define    EPS                  10E-8
#define    V_TAG                99
#define    P_TAG                100
#define    Q_TAG                101
#define    R_TAG                102
#define    S_TAG                103
#define    S_TAG2               104

void evaluate(complex<double>* f, int beginPos,
int endPos, const complex<double>* x,
        complex<double> *y, int leftPos,
        int rightPos, int totalLength);

void shuffle(complex<double>* f, int beginPos,
       int endPos);

void print(const complex<double>* f,
        int fLength);

void readDoubleComplex(FILE *f,
        complex<double> &z);

int main(int argc, char *argv[])
{
        complex<double> p[MAX_N], q[MAX_N],
        s[2*MAX_N], r[2*MAX_N];
        complex<double> w[2*MAX_N];
        complex<double> temp;
        int variableNum;
        MPI_Status status;
        int rank, size;
        int i, j, k, n;
        int wLength;
        int everageLength;
        int moreLength;
        int startPos;
        int stopPos;
        FILE *fin;

       MPI_Init(&argc, &argv);
       MPI_Get_rank(MPI_COMM_WORLD,
               &rank);

        MPI_Get_size(MPI_COMM_WORLD,
                &size);

       if(rank == 0)
       {
               fin = fopen("dataIn.txt", "r");
               if (fin == NULL)
               {
                       puts("Not find input data file");
                       puts("Please create a file
```

```
                \"dataIn.txt\"");
            puts("<example for dataIn.txt> ");
            puts("2");
            puts("1.0    2");
            puts("2.0    -1");
            exit(-1);
        }

        readDoubleComplex(fin, variableNum);

        if ((variableNum < 1)||(variableNum >
            MAX_N))
        {
            puts("variableNum out of range!");
            exit(-1);
        }

        for(i = 0; i < variableNum; i ++)
            readDoubleComplex(fin, p[i]);
        for(i = 0; i < variableNum; i ++)
            readDoubleComplex(fin, q[i]);
        fclose(fin);

        puts("Read from data file \"dataIn.txt\"");
        printf("p(t) = ");
        print(p, variableNum);
        printf("q(t) = ");
        print(q, variableNum);

        for(i = 1; i < size; i ++)
        {
            MPI_Send(&variableNum,1,
                MPI_INT,i, V_TAG,
                MPI_COMM_WORLD);
            MPI_Send(p,variableNum,
                MPI_DOUBLE_COMPLEX,i,
                P_TAG,
                PI_COMM_WORLD);
                 MPI_Send(q,variableNum,
                MPI_DOUBLE_COMPLEX,i,
                Q_TAG,
                MPI_COMM_WORLD);
        }
}
else
{
        MPI_Recv(&variableNum,1,MPI_INT,0,
                V_TAG,MPI_COMM_WORLD,
                &status);
        MPI_Recv(p,variableNum,
                MPI_DOUBLE_COMPLEX,0,
                P_TAG, PI_COMM_WORLD,
                &status);
       MPI_Recv(q,variableNum,
                MPI_DOUBLE_COMPLEX,0,
                Q_TAG,MPI_COMM_WORLD,
                &status);
 }

wLength = 2*variableNum;
for(i = 0; i < wLength; i ++)
{
        w[i]= complex<double>
                (cos(i*2*PI/wLength),
                sin(i*2*PI/wLength));
}

everageLength = wLength / size;
moreLength = wLength % size;
startPos = moreLength + rank * everageLength;
stopPos = startPos + everageLength - 1;

if(rank == 0)
{
         startPos = 0;
         stopPos  =  moreLength+everageLength  -
                1;
}
evaluate(p, 0, variableNum - 1, w, s,
        startPos, stopPos, wLength);
evaluate(q, 0, variableNum - 1, w, r,
        startPos, stopPos, wLength);
for(i = startPos; i <= stopPos ; i ++)
        s[i] = s[i]*r[i]/(wLength*1.0);

if (rank > 0)
{
        MPI_Send((s+startPos),
```

```
                everageLength,
                MPI_DOUBLE_COMPLEX, 0,
                S_TAG, MPI_COMM_WORLD);
        MPI_Recv(s,wLength,
                MPI_DOUBLE_COMPLEX,0,
                S_ TAG2,MPI_ COMM_WORLD,
                &status);
}
else
{
        for(i = 1; i < size; i ++)
        {
                MPI_Recv((s+moreLength+i*
                        everageLength),everageLength,
                        MPI_DOUBLE_COMPLEX,
                        i,S_TAG,
                        MPI_COMM_WORLD,
                        &status);
        }

        for(i = 1; i < size; i ++)
        {
                MPI_Send(s,wLength,
                        MPI_DOUBLE_COMPLEX,i,
                        S_TAG2,
                        MPI_COM M_WORLD);
        }
}

for(int i = 1; i < wLength/2; i ++)
{
        temp = w[i];
        w[i] = w[wLength - i];
        w[wLength - i] = temp;
}
evaluate(s, 0, wLength - 1, w, r, startPos,
        stopPos, wLength);

if (rank > 0)
{
        MPI_Send((r+startPos), everageLength,
                MPI_DOUBLE_COMPLEX,0,
                R_TAG,
                MPI_COMM_WORLD);
}
else
{
        for(i = 1; i < size; i ++)
        {
                MPI_Recv((r+moreLength+i*
                        everageLength),
                        everageLength,
                        MPI_DOUBLE_COMPLEX,
                        i, R_TAG,
                        MPI_COMM_WORLD,
                        &status);
        }

        puts("\nAfter FFT r(t)=p(t)q(t)");
        printf("r(t) = ");
        print(r, wLength - 1);
        puts("");
        printf("Use prossor size = %d\n",size);
}

MPI_Finalize();
}

void evaluate(complex<double>* f, int beginPos, int
        endPos, const complex<double>* x,
        complex<double>* y, int leftPos, int
        rightPos, int totalLength)
{
        int i;
        complex<double>
                tempX[2*MAX_N],tempY1[2*MAX_N],
                tempY2[2*MAX_N];
        int midPos = (beginPos + endPos)/2;

        if ((beginPos > endPos)||(leftPos > rightPos))
        {
                puts("Error in use Polynomial!");
                exit(-1);
        }
        else if(beginPos == endPos)
        {
                for(i = leftPos; i <= rightPos; i ++)
                        y[i] = f[beginPos];
```

```
        }
        else if(beginPos + 1 == endPos)
        {
            for(i = leftPos; i <= rightPos; i ++)
                y[i] = f[beginPos] + f[endPos]*x[i];
        }
        else
        {
            shuffle(f, beginPos, endPos);
            for(i = leftPos; i <= rightPos; i ++)
                tempX[i] = x[i]*x[i];
            evaluate(f, beginPos, midPos, tempX,
                tempY1, leftPos, rightPos,totalLength);
            evaluate(f, midPos+1, endPos, tempX,
                tempY2, leftPos, rightPos,
                totalLength);
            for(i = leftPos; i <= rightPos; i ++)
                y[i] = tempY1[i] + x[i]*tempY2[i];
        }
}

void shuffle(complex<double>* f, int beginPos, int
        endPos)
{
        complex<double> temp[2*MAX_N];
        int i, j;

        for(i = beginPos; i <= endPos; i ++)
            temp[i] = f[i];

        j = beginPos;
        for(i = beginPos; i <= endPos; i +=2)
        {
            f[j] = temp[i];
            j ++;
        }
        for(i = beginPos +1; i <= endPos; i += 2)
        {
            f[j] = temp[i];
            j ++;
        }
}

void print(const complex<double>* f, int fLength)
{
        bool isPrint = false;
        int i;

        if (abs(f[0].real()) > EPS)
        {
            printf("%lf", f[0].real());
            isPrint = true;
        }

        for(i = 1; i < fLength; i ++)
        {
            if (f[i].real() > EPS)
            {
                if (isPrint) printf(" + ");
                else isPrint = true;
                printf("%lft^%d", f[i].real(),i);
            }
            else if (f[i].real() < - EPS)
            {
                if(isPrint) printf(" - ");
                else isPrint = true;
                printf("%lft^%d", -f[i].real(),i);
            }
        }

        if (isPrint == false) printf("0");
        printf("\n");
}
```

2. 运行实例

编译： mpicc –o fft fft.cc

运行： 使用如下命令运行程序

mpirun –np 1 fft

mpirun –np 2 fft

mpirun –np 3 fft

mpirun –np 4 fft

mpirun –np 5 fft

运行结果：

```
Input of file "dataIn.txt"
4
1   3   3   1
0   1   2   1

Output of solution
    Read from data file "dataIn.txt"
    p(t) = 1 + 3t^1 + 3t^2 + 1t^3
    q(t) = 1t^1 + 2t^2 + 1t^3

    After FFT r(t)=p(t)q(t)
    r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

    Use prossor size = 1
    End of this running

    Read from data file "dataIn.txt"
    p(t) = 1 + 3t^1 + 3t^2 + 1t^3
    q(t) = 1t^1 + 2t^2 + 1t^3

    After FFT r(t)=p(t)q(t)
    r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

    Use prossor size = 2
    End of this running

    Read from data file "dataIn.txt"
    p(t) = 1 + 3t^1 + 3t^2 + 1t^3
    q(t) = 1t^1 + 2t^2 + 1t^3

    After FFT r(t)=p(t)q(t)
    r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

    Use prossor size = 3
    End of this running

    Read from data file "dataIn.txt"
    p(t) = 1 + 3t^1 + 3t^2 + 1t^3
    q(t) = 1t^1 + 2t^2 + 1t^3

    After FFT r(t)=p(t)q(t)
```

```
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 4
End of this running

Read from data file "dataIn.txt"
p(t) = 1 + 3t^1 + 3t^2 + 1t^3
q(t) = 1t^1 + 2t^2 + 1t^3

After FFT r(t)=p(t)q(t)
r(t) = 1t^1 + 5t^2 + 10t^3 + 10t^4 + 5t^5 + 1t^6

Use prossor size = 5
End of this running
```

说明：运行中可以使用参数 ProcessSize，如 mpirun －np ProcessSize fft 来运行该程序，其中 ProcessSize 是所使用的处理器个数，本实例中依次取 1、2、3、4、5 个处理器分别进行计算。